# Resumen de la sesión 4

## Spark: de los Parquet a las consultas




**Curso 2026/2027**

Diego Sevilla
Departamento de Ingeniería y Tecnología de Computadores
Facultad de Informática · Universidad de Murcia

dsevilla@um.es

# Qué vamos a repasar

La sesión 4 pone a **Spark** a trabajar sobre los Parquet de S2. Este resumen recorre el camino que sigue una consulta, de la lectura al reparto del trabajo:

- **DataFrames y expresiones**: leer, seleccionar, filtrar y calcular columnas, y entender por qué describir un cálculo no es ejecutarlo
- **Agrupar y unir**: qué representa cada fila antes y después de un `groupBy` o de un `join`
- **Spark SQL**: las mismas operaciones expresadas con vistas y consultas
- **Cómo se ejecuta**: el plan, las particiones, el *shuffle*, YARN y qué conviene guardar

> Quedan fuera de este resumen, pero están en el cuaderno: funciones de ventana, `rollup` y `cube`, tipos de join menos comunes, arrays y structs, y `spark-submit`

# Los mismos datos, otro motor

S2 dejó las tablas TPC-DS como ficheros Parquet en HDFS, bajo `/datalake/raw/tpcds`. Spark lee **esos mismos ficheros**: no hace falta generar los datos otra vez ni cargarlos en una base de datos.

- Un **DataFrame** representa datos con columnas y tipos, junto con el plan para obtenerlos
- Se consulta con la API de DataFrames o con SQL, y las dos pasan por el mismo motor
- Primero se ejecuta en local, en el propio equipo del driver; después, YARN reparte las tareas entre ejecutores que corren en los DataNodes

> Cambia dónde se hace el cálculo; el significado de la consulta debe conservarse

# DataFrames y expresiones

Leer, seleccionar, filtrar y calcular columnas

# Leer Parquet y conocer el esquema

Con la sesión `spark` del cuaderno ya abierta:

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F

web_sales: DataFrame = spark.read.parquet("/datalake/raw/tpcds/web_sales")
item: DataFrame = spark.read.parquet("/datalake/raw/tpcds/item")
web_sales.printSchema()
```

Se lee un **directorio** de Parquet, no un fichero: Spark toma el esquema de los propios ficheros, y `printSchema()` muestra el nombre, el tipo y si admite nulos de cada columna.

Recuerda que una fila de `web_sales` es una **línea de venta**: un pedido con tres artículos ocupa tres filas.

# Describir el cálculo todavía no lo ejecuta

```
pérdidas: DataFrame = web_sales.filter(
    F.col("ws_net_profit") < 0
).select("ws_order_number", "ws_net_profit")
```

Esta celda termina al instante y no muestra nada. Spark sólo ha **anotado en un plan** que queremos las líneas con beneficio negativo y dos de sus columnas. La variable `pérdidas` es un DataFrame, es decir, una descripción del cálculo, no una lista de filas traída a Python.

Mientras sólo encadenemos operaciones de este tipo, no se lee ningún dato de las ventas. Spark espera a que pidamos un resultado para decidir cómo calcularlo de la forma más económica.

> Las transformaciones describen qué calcular; una acción pide obtener un resultado

# Transformaciones y acciones

| Construyen otro plan | Ejecutan para obtener un resultado |
|---|---|
| `select`, `filter`, `withColumn` | `show(n)`: muestra unas filas |
| `groupBy(...).agg(...)`, `join` | `count()`: devuelve el número de filas |
| `orderBy`, `limit`, `spark.sql(...)` | `collect()`: trae todas las filas al driver |
| | `write.parquet(...)`: escribe el resultado |

Una acción ejecuta el plan, pero no siempre recorre todo. `perdidas.show(5)` puede parar en cuanto encuentra cinco filas que cumplen el filtro; en cambio `perdidas.count()` debe revisar todos los datos, porque necesita el total. Si hay pocas coincidencias, incluso `show(5)` tendrá que buscar en muchas particiones.

Reserva `collect()` y `toPandas()` para resultados pequeños: traen todo a la memoria del driver.

# Expresiones de columna

```
ventas = web_sales.select(
    "ws_order_number", F.col("ws_net_profit").alias("beneficio"),
    F.year("ws_sold_timestamp").alias("sold_year"))
con_perdida = web_sales.filter(
    F.col("ws_sold_timestamp").isNotNull() & (F.col("ws_net_profit") < 0))
clasificadas = web_sales.withColumn("resultado",
    F.when(F.col("ws_net_profit").isNull(), "desconocido")
     .when(F.col("ws_net_profit") < 0, "pérdida").otherwise("sin pérdida"))
```

- Ninguna de las tres modifica `web_sales`: cada una devuelve **otro DataFrame**. No recorremos filas con un bucle Python; describimos el cálculo y Spark lo reparte
- Con columnas se usan `&`, `|` y `~`, siempre entre paréntesis; las palabras `and`, `or` y `not` de Python no funcionan
- `filter` conserva las filas donde la condición es verdadera, y una comparación con `null` nunca lo es: por eso `isNull()` e `isNotNull()` se tratan aparte
- Conviene preferir las funciones integradas (`F.when`, `F.year`…): Spark las entiende y las optimiza. Una UDF de Python sirve para casos propios, pero añade coste y esconde su lógica al optimizador

# Agrupar y unir

Qué representa cada fila antes y después del cálculo

# `groupBy` y `agg`: beneficio por año

```
por_año: DataFrame = (
    web_sales.filter(F.col("ws_sold_timestamp").isNotNull())
    .groupBy(F.year("ws_sold_timestamp").alias("sold_year"))
    .agg(
        F.sum("ws_net_profit").alias("beneficio_neto"),
        F.countDistinct("ws_order_number").alias("pedidos"),
        F.count("*").alias("lineas_de_venta"),
    )
    .orderBy("sold_year")
)
por_año.show()
```

Antes del cálculo hay **una fila por línea de venta**; después, **una fila por año**. `groupBy` decide los grupos y `agg` calcula las medidas dentro de cada uno. Antes de escribir una agregación conviene preguntarse qué representa una fila del resultado.

# Contar filas no es contar pedidos

Tres líneas de venta de dos pedidos (ejemplo ilustrativo):

| Pedido | Artículo | Beneficio neto |
|---|---|---|
| 100 | A | 8 |
| 100 | B | `null` |
| 101 | C | −2 |

| Expresión dentro de `agg` | Resultado | Qué significa |
|---|---|---|
| `F.count("*")` | 3 | Líneas de venta |
| `F.countDistinct("ws_order_number")` | 2 | Pedidos distintos |
| `F.count("ws_net_profit")` | 2 | Líneas con beneficio conocido |
| `F.sum("ws_net_profit")` | 6 | Suma de los beneficios conocidos |

El `null` no cuenta como cero: `count("ws_net_profit")` lo ignora y `sum` lo salta. Y no hay que confundir `F.count(...)`, que construye una expresión dentro de `agg`, con `df.count()`, que ejecuta un recuento y devuelve un entero.

# Filtrar grupos y ordenar un resultado

Clientes conocidos con más de cinco pedidos, de mayor a menor actividad:

```
clientes_activos: DataFrame = (
    web_sales.filter(F.col("ws_bill_customer_sk").isNotNull())
    .groupBy("ws_bill_customer_sk")
    .agg(F.countDistinct("ws_order_number").alias("pedidos"))
    .filter(F.col("pedidos") > 5)
    .orderBy(F.col("pedidos").desc(), "ws_bill_customer_sk")
    .limit(10)
)
```

Hay dos `filter` con papeles distintos. El primero se aplica **antes** de agrupar y elige líneas; el segundo, **después** de `agg`, elige grupos, igual que `HAVING` en SQL.

`orderBy` fija el orden y el desempate, para que el resultado sea repetible. `limit` recorta el resultado, pero no evita calcular todos los grupos antes.

# `join`: añadir la categoría de cada venta

```
ventas_categoria: DataFrame = (
    web_sales.join(item, web_sales.ws_item_sk == item.i_item_sk, "inner")
    .select(F.year("ws_sold_timestamp").alias("sold_year"),
            "i_category", "ws_net_profit")
)
ventas_categoria.groupBy("sold_year", "i_category").agg(
    F.sum("ws_net_profit").alias("beneficio_neto")
).orderBy("sold_year", "i_category").show()
```

La condición `ws_item_sk = i_item_sk` enlaza cada línea de venta con la versión del artículo que se vendió. Con `inner` sólo se conservan las ventas que encuentran pareja; con `left` se conservarían todas, con la categoría vacía en las que no la encuentran.

**Cuidado:** si una clave encuentra varias filas al otro lado, la venta se repite una vez por cada coincidencia, y una suma posterior inflaría el importe. Las claves `_sk` sirven para unir, no son nombres de clientes ni de provincias.

# Vistas y Spark SQL

Las mismas operaciones, expresadas con SQL

# Dar un nombre SQL a un DataFrame

```
web_sales.createOrReplaceTempView("web_sales")

muestra: DataFrame = spark.sql("""
    SELECT ws_order_number, ws_net_profit
    FROM web_sales
    WHERE ws_net_profit < 0
""")
muestra.show(5)
```

`createOrReplaceTempView` asocia un nombre SQL al DataFrame. La vista pertenece a la **sesión de Spark**: no crea una tabla persistente ni copia los Parquet, igual que las vistas de DuckDB en S2.

`spark.sql(...)` devuelve a su vez un DataFrame, así que podemos seguir transformándolo con la API. SQL y DataFrames son dos formas de pedir lo mismo al mismo planificador.

# La agregación anual, ahora en SQL

```
por_año_sql: DataFrame = spark.sql("""
    SELECT year(ws_sold_timestamp) AS sold_year,
           sum(ws_net_profit) AS beneficio_neto,
           count(DISTINCT ws_order_number) AS pedidos,
           count(*) AS lineas_de_venta
    FROM web_sales
    WHERE ws_sold_timestamp IS NOT NULL
    GROUP BY year(ws_sold_timestamp)
    ORDER BY sold_year
""")
por_año_sql.show()
```

Es el mismo filtro, los mismos grupos y las mismas medidas que `por_año`, y debe dar los mismos valores tanto en local como sobre YARN: cambia dónde se calcula, no el resultado.

> Para pensar: ¿por qué en cada año `pedidos` es menor que `lineas_de_venta`? ¿Qué pasaría si sumáramos `ws_ext_sales_price` en lugar de `ws_net_profit`?

# Cómo se ejecuta

Leer el plan y reconocer dónde se mueve el dato

# `explain`: mirar el trabajo que hemos pedido

```
web_sales.select("ws_order_number", "ws_net_profit") \
    .filter(F.col("ws_net_profit") > 0).explain("formatted")
```

```
(1) Scan parquet
Output [2]: [ws_order_number, ws_net_profit]
PushedFilters: [IsNotNull(ws_net_profit), GreaterThan(ws_net_profit,0)]
ReadSchema: struct<ws_order_number:bigint,ws_net_profit:decimal(7,2)>
(3) Filter   Condition : (isnotnull(ws_net_profit) AND (ws_net_profit > 0))
```

`ReadSchema` dice que sólo se leen **dos columnas** de las que tiene `web_sales`, y `PushedFilters` que el filtro se entrega al lector de Parquet para descartar datos pronto. Salida abreviada. En planes con agregaciones o uniones aparecerá también `Exchange`, una redistribución de datos entre particiones.

`explain` no calcula el resultado; para ver tiempos y tareas se mira la Spark UI mientras se ejecuta una acción.

# Particiones y *shuffle*

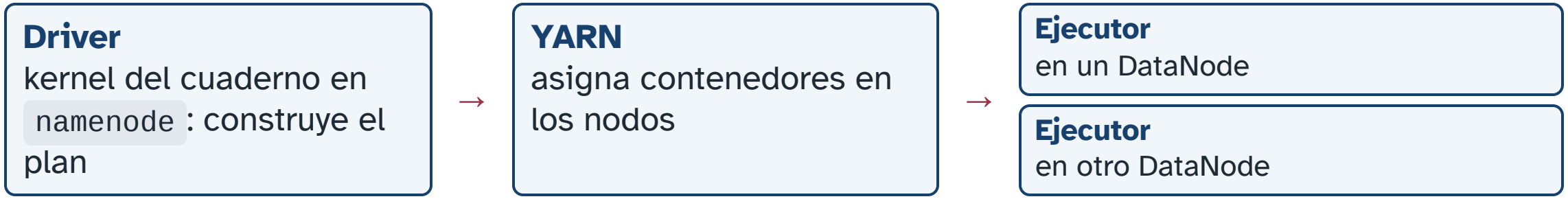
Una **partición de ejecución** es el trozo de un DataFrame que procesa una tarea. Operaciones como `select` o `filter` trabajan sobre cada partición por separado, sin necesitar filas de las demás.

Una agregación por clave es distinta: las filas de una misma clave pueden estar repartidas en muchas particiones, y hay que reunirlas antes de sumar. Ese intercambio de datos entre particiones, a través de la red, es el ***shuffle***, y suele ser lo más caro de una consulta.

Un `join` puede obligar a redistribuir los dos lados. Si uno de ellos es pequeño, un *broadcast join* lo copia entero a cada ejecutor y evita mover el grande.

Un fichero Parquet, un bloque de HDFS y una partición de ejecución **no son lo mismo**, y tener tres ejecutores no garantiza tres partes iguales: importa el reparto real de las filas.

# Driver, ejecutores y YARN




En `local[2]` todo ocurre en el equipo del driver, con dos hilos. Al cambiar el `master` a YARN, **el driver sigue en `namenode`**, pero los ejecutores son procesos reales en los DataNodes, que leen los Parquet de HDFS.

Una acción lanza un trabajo (*job*), dividido en etapas (*stages*) y tareas (*tasks*), y cada tarea procesa una partición. El informe de hosts de S4 muestra en qué DataNode se procesó cada partición y cuántas filas tenía.

# Reutilizar con `cache` y repartir con `repartition`

- `cache()` marca un DataFrame para guardarlo en memoria cuando una acción lo calcule. Las acciones siguientes lo reutilizan en vez de recalcularlo desde HDFS; `unpersist()` libera esa memoria. Sólo compensa si se usa **más de una vez**
- `repartition(n)` redistribuye las filas en `n` particiones mediante un *shuffle* completo. Es útil, por ejemplo, para equilibrar el trabajo antes de un `join`
- `coalesce(n)` reduce el número de particiones sin un *shuffle* completo: es más barato, pero puede dejar particiones desiguales

Con `getNumPartitions()` se comprueba cuántas particiones tiene un DataFrame antes y después de cada operación.

> Ninguna de las tres cambia el resultado de la consulta, sólo cómo se reparte y se reutiliza el trabajo

# Guardar un resultado

```
por_año.coalesce(1).write.mode("overwrite").parquet(
    "/user/luser/s4/resumen-beneficio-anual"
)
```

La escritura crea un **directorio** con uno o varios ficheros Parquet, no un único fichero: Spark escribe un fichero por partición. `mode("overwrite")` sustituye lo que hubiera en esa ruta.

`por_año` tiene una fila por año, así que `coalesce(1)` evita dejar muchos ficheros diminutos, el problema de los **ficheros pequeños**, que ralentiza las lecturas posteriores.

Reducir a una sola partición sólo es apropiado para resultados pequeños. Con un resultado grande, una única tarea tendría que escribir todo y se perdería el paralelismo.

# Lo que hay que llevarse de S4

- Spark lee los **mismos Parquet** de S2; un DataFrame es una descripción del cálculo, y sólo una **acción** lo ejecuta
- Antes de escribir una consulta hay que saber qué representa una fila, qué mide cada columna y por qué clave se agrupa: `count(*)` cuenta líneas, `countDistinct(pedido)` cuenta pedidos, y una suma de ventas no es una suma de beneficios
- Una unión puede **multiplicar filas**, y los `null` no son ceros
- La API de DataFrames y SQL dicen lo mismo; `explain` enseña cómo intenta resolverlo Spark y dónde hay **shuffle**
- En YARN el driver sigue en `namenode` y los ejecutores trabajan en los DataNodes: se reparten **tareas sobre particiones**, no filas por igual
- Un resultado se guarda como directorio de Parquet; conviene evitar los ficheros pequeños

> **En S5** Spark deja de leer sólo `raw` y escribe datos *silver* y un producto *gold*, con convenciones de nombres, controles de calidad y particionado físico